Operación Cielos Limpios

Herencia en Java

Autor: Isac Martinez

Proyecto: CielosLimpios

GitHub: https://github.com/isacmn45/CielosLimpios

Clases: Dron, DronReconocimientoTactico, DronCargaAmbiental y Main.

Dron.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this
license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package com.mycompany.cieloslimpios;

/**
 * Chasis común de todas las unidades de la operación Cielos Limpios.
 *
 * @author isacmn45
 */
public class Dron {

    private String codigoSerie;
    private int nivelEnergia;
    private boolean vinculadoRed;

    public Dron(String codigoSerie, int nivelEnergia, boolean vinculadoRed) {
        setCodigoSerie(codigoSerie);
        setNivelEnergia(nivelEnergia);
        this.vinculadoRed = vinculadoRed;
    }

    public String getCodigoSerie() {
        return codigoSerie;
    }

    public void setCodigoSerie(String codigoSerie) {
        if (codigoSerie == null || codigoSerie.trim().isEmpty()) {
            throw new IllegalArgumentException("El código de serie no puede estar vacío.");
        }
        this.codigoSerie = codigoSerie.trim();
    }

    public int getNivelEnergia() {
        return nivelEnergia;
    }

    /** Los valores irregulares se ajustan al porcentaje válido más cercano. */
    public void setNivelEnergia(int nivelEnergia) {
        this.nivelEnergia = Math.max(0, Math.min(100, nivelEnergia));
        if (this.nivelEnergia != nivelEnergia) {
            System.out.printf("[%s] Energía solicitada: %d%%; ajustada a %d%%.%n",
                    codigoSerie, nivelEnergia, this.nivelEnergia);
        }
        calibrarGiroscopioInterno();
    }

    public boolean isVinculadoRed() {
        return vinculadoRed;
    }

    public void setVinculadoRed(boolean vinculadoRed) {
        this.vinculadoRed = vinculadoRed;
    }

    // Private: ni el operador ni las subclases pueden invocar este protocolo.
    private void calibrarGiroscopioInterno() {
        System.out.printf("[%s] Sensores internos reajustados: giroscopio calibrado.%n",
                codigoSerie);
    }

    public void transmitirEstadoRed() {
        System.out.printf("[BALIZAS] Serie: %s | Batería: %d%% | En línea: %s%n",
                codigoSerie, nivelEnergia, vinculadoRed ? "sí" : "no");
    }
}
```

DronReconocimientoTactico.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this
license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package com.mycompany.cieloslimpios;

import java.util.Objects;
import java.util.Random;

/**
 * Unidad especializada en patrullar los siete cuadrantes forestales.
 *
 * @author isacmn45
 */
public class DronReconocimientoTactico extends Dron {

    private final int[] lecturasTermicas = new int[7];
    private final Random random;

    public DronReconocimientoTactico(String codigoSerie, int nivelEnergia,
            boolean vinculadoRed) {
        this(codigoSerie, nivelEnergia, vinculadoRed, new Random());
    }

    // Permite verificar los umbrales con lecturas reproducibles en las pruebas.
    DronReconocimientoTactico(String codigoSerie, int nivelEnergia,
            boolean vinculadoRed, Random random) {
        super(codigoSerie, nivelEnergia, vinculadoRed);
        this.random = Objects.requireNonNull(random, "El generador es obligatorio.");
    }

    public int[] getLecturasTermicas() {
        return lecturasTermicas.clone();
    }

    @Override
    public void transmitirEstadoRed() {
        super.transmitirEstadoRed();
        System.out.printf("[%s] Modelo: Reconocimiento táctico | Cuadrantes: %d%n",
                getCodigoSerie(), lecturasTermicas.length);
    }

    public void generarMapeoEstocastico() {
        System.out.printf("%n[%s] Escaneando los siete cuadrantes:%n", getCodigoSerie());
        for (int i = 0; i < lecturasTermicas.length; i++) {
            lecturasTermicas[i] = random.nextInt(51); // Incluye 0 y 50.
            System.out.printf("  Sector %d: amenaza térmica %d.%n",
                    i + 1, lecturasTermicas[i]);
        }
    }

    public void analizarAmenazas() {
        for (int i = 0; i < lecturasTermicas.length; i++) {
            if (lecturasTermicas[i] > 35) {
                System.out.printf("[%s] Riesgo detectado en el sector %d: %d.%n",
                        getCodigoSerie(), i + 1, lecturasTermicas[i]);
                emitirAlertaIncendio();
                return; // Se detiene este análisis ante el primer peligro.
            }
        }
        System.out.printf("[%s] Patrulla completada sin amenazas superiores a 35.%n",
                getCodigoSerie());
    }

    private void emitirAlertaIncendio() {
        System.out.printf("[%s] ¡ALERTA ROJA! Posible incendio en el sector%n",
                getCodigoSerie());
    }
}
```

DronCargaAmbiental.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this
license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package com.mycompany.cieloslimpios;

import java.util.Locale;

/**
 * Unidad recolectora de desechos; todos los pesos se expresan en kilogramos.
 *
 * @author isacmn45
 */
public class DronCargaAmbiental extends Dron {

    private final double cargaMaxima;
    private double cargaActual;

    public DronCargaAmbiental(String codigoSerie, int nivelEnergia,
            boolean vinculadoRed, double cargaMaxima) {
        super(codigoSerie, nivelEnergia, vinculadoRed);
        if (!Double.isFinite(cargaMaxima) || cargaMaxima <= 0) {
            throw new IllegalArgumentException("La carga máxima debe ser positiva y finita.");
        }
        this.cargaMaxima = cargaMaxima;
        this.cargaActual = 0;
    }

    public double getCargaMaxima() {
        return cargaMaxima;
    }

    public double getCargaActual() {
        return cargaActual;
    }

    @Override
    public void transmitirEstadoRed() {
        super.transmitirEstadoRed();
        System.out.printf(Locale.ROOT,
                "[%s] Modelo: Carga ambiental | Carga: %.2f / %.2f kg.%n",
                getCodigoSerie(), cargaActual, cargaMaxima);
    }

    /** Retorna true si el objeto provocaría una sobrecarga; no modifica el dron. */
    public boolean evaluarPesoObjeto(double pesoNuevo) {
        if (!Double.isFinite(pesoNuevo) || pesoNuevo < 0) {
            throw new IllegalArgumentException("El peso debe ser finito y no negativo.");
        }
        return cargaActual + pesoNuevo > cargaMaxima;
    }

    public void engancharDesecho(double pesoNuevo) {
        if (evaluarPesoObjeto(pesoNuevo)) {
            System.out.printf(Locale.ROOT,
                    "[%s] Maniobra abortada: objeto de %.2f kg demasiado pesado. "
                    + "Carga: %.2f / %.2f kg.%n",
                    getCodigoSerie(), pesoNuevo, cargaActual, cargaMaxima);
        } else {
            cargaActual += pesoNuevo;
            System.out.printf(Locale.ROOT,
                    "[%s] Recolección exitosa: %.2f kg. Carga: %.2f / %.2f kg.%n",
                    getCodigoSerie(), pesoNuevo, cargaActual, cargaMaxima);
        }
    }

    public void descargarContenedorBase() {
        System.out.printf(Locale.ROOT, "[%s] Descarga en la base: %.2f kg liberados.%n",
                getCodigoSerie(), cargaActual);
        cargaActual = 0;
    }
}
```

Main.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this
license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package com.mycompany.cieloslimpios;

import java.util.Locale;
import java.util.Random;

/**
 * Consola central de la operación: las dos flotas se gestionan por separado.
 *
 * @author isacmn45
 */
public class Main {

    public static void main(String[] args) {
        Random random = new Random();
        System.out.println("=== OPERACIÓN CIELOS LIMPIOS | AÑO 2030 ===");

        DronReconocimientoTactico[] escuadronTactico = {
            new DronReconocimientoTactico("TAC-001", 100, true),
            new DronReconocimientoTactico("TAC-002", 80, true),
            new DronReconocimientoTactico("TAC-003", 65, true)
        };

        DronCargaAmbiental[] escuadronPesado = {
            new DronCargaAmbiental("CAR-001", 90, true, 20.0),
            new DronCargaAmbiental("CAR-002", 60, true, 35.0)
        };

        System.out.println("\n--- MISIÓN DEL ESCUADRÓN TÁCTICO ---");
        for (DronReconocimientoTactico dron : escuadronTactico) {
            dron.generarMapeoEstocastico();
            dron.analizarAmenazas();
        }

        System.out.println("\n--- MISIÓN DEL ESCUADRÓN PESADO ---");
        for (DronCargaAmbiental dron : escuadronPesado) {
            double pesoNuevo = 1 + random.nextInt(50); // De 1 a 50 kg.
            System.out.printf(Locale.ROOT, "[%s] Desecho encontrado: %.2f kg.%n",
                    dron.getCodigoSerie(), pesoNuevo);
            dron.engancharDesecho(pesoNuevo);
            dron.descargarContenedorBase();
        }

        System.out.println("\n--- PRUEBA DEL SEGURO DE ENERGÍA SUPERIOR ---");
        escuadronTactico[0].setNivelEnergia(125); // Se limita a 100%.

        System.out.println("\n--- DESGASTE POST-MISIÓN Y TELEMETRÍA FINAL ---");
        // El consumo de la segunda unidad supera su batería para probar el límite 0.
        int[] desgasteTactico = {15, 120, 10};
        for (int i = 0; i < escuadronTactico.length; i++) {
            DronReconocimientoTactico dron = escuadronTactico[i];
            dron.setNivelEnergia(dron.getNivelEnergia() - desgasteTactico[i]);
            dron.setVinculadoRed(dron.getNivelEnergia() > 0);
            dron.transmitirEstadoRed();
        }

        int[] desgastePesado = {20, 100};
        for (int i = 0; i < escuadronPesado.length; i++) {
            DronCargaAmbiental dron = escuadronPesado[i];
            dron.setNivelEnergia(dron.getNivelEnergia() - desgastePesado[i]);
            dron.setVinculadoRed(dron.getNivelEnergia() > 0);
            dron.transmitirEstadoRed();
        }

        System.out.println("\n=== SUPERVISIÓN FINALIZADA ===");
    }

}
```

EVIDENCIAS

Output - CielosLimpios (run)

```
ant -f /home/isacmn45/NetBeansProjects/CielosLimpios run
init:
compile:
run:
=== OPERACIÓN CIELOS LIMPIOS | AÑO 2030 ===
[TAC-001] Sensores internos reajustados: giroscopio calibrado.
[TAC-002] Sensores internos reajustados: giroscopio calibrado.
[TAC-003] Sensores internos reajustados: giroscopio calibrado.
[CAR-001] Sensores internos reajustados: giroscopio calibrado.
[CAR-002] Sensores internos reajustados: giroscopio calibrado.

--- MISIÓN DEL ESCUADRÓN TÁCTICO ---

[TAC-001] Escaneando los siete cuadrantes:
  Sector 1: amenaza térmica 44.
  Sector 2: amenaza térmica 48.
  Sector 3: amenaza térmica 27.
  Sector 4: amenaza térmica 39.
  Sector 5: amenaza térmica 0.
  Sector 6: amenaza térmica 39.
  Sector 7: amenaza térmica 7.
[TAC-001] Riesgo detectado en el sector 1: 44.
[TAC-001] ¡ALERTA ROJA! Posible incendio en el sector

[TAC-002] Escaneando los siete cuadrantes:
  Sector 1: amenaza térmica 8.
  Sector 2: amenaza térmica 29.
  Sector 3: amenaza térmica 45.
  Sector 4: amenaza térmica 23.
  Sector 5: amenaza térmica 18.
  Sector 6: amenaza térmica 36.
  Sector 7: amenaza térmica 39.
[TAC-002] Riesgo detectado en el sector 3: 45.
[TAC-002] ¡ALERTA ROJA! Posible incendio en el sector

[TAC-003] Escaneando los siete cuadrantes:
  Sector 1: amenaza térmica 35.
  Sector 2: amenaza térmica 11.
  Sector 3: amenaza térmica 3.
  Sector 4: amenaza térmica 8.
  Sector 5: amenaza térmica 30.
  Sector 6: amenaza térmica 1.
  Sector 7: amenaza térmica 28.
[TAC-003] Patrulla completada sin amenazas superiores a 35.
```

Output - CielosLimpios (run) ×

```
[TAC-002] ¡ALERTA ROJA: Posible incendio en el sector

[TAC-003] Escaneando los siete cuadrantes:
  Sector 1: amenaza térmica 35.
  Sector 2: amenaza térmica 11.
  Sector 3: amenaza térmica 3.
  Sector 4: amenaza térmica 8.
  Sector 5: amenaza térmica 30.
  Sector 6: amenaza térmica 1.
  Sector 7: amenaza térmica 28.
[TAC-003] Patrulla completada sin amenazas superiores a 35.

--- MISIÓN DEL ESCUADRÓN PESADO ---
[CAR-001] Desecho encontrado: 23.00 kg.
[CAR-001] Maniobra abortada: objeto de 23.00 kg demasiado pesado. Carga: 0.00 / 20.00 kg.
[CAR-001] Descarga en la base: 0.00 kg liberados.
[CAR-002] Desecho encontrado: 1.00 kg.
[CAR-002] Recolección exitosa: 1.00 kg. Carga: 1.00 / 35.00 kg.
[CAR-002] Descarga en la base: 1.00 kg liberados.

--- PRUEBA DEL SEGURO DE ENERGÍA SUPERIOR ---
[TAC-001] Energía solicitada: 125%; ajustada a 100%.
[TAC-001] Sensores internos reajustados: giroscopio calibrado.

--- DESGASTE POST-MISIÓN Y TELEMETRÍA FINAL ---
[TAC-001] Sensores internos reajustados: giroscopio calibrado.
[BALIZAS] Serie: TAC-001 | Batería: 85% | En línea: sí
[TAC-001] Modelo: Reconocimiento táctico | Cuadrantes: 7
[TAC-002] Energía solicitada: -40%; ajustada a 0%.
[TAC-002] Sensores internos reajustados: giroscopio calibrado.
[BALIZAS] Serie: TAC-002 | Batería: 0% | En línea: no
[TAC-002] Modelo: Reconocimiento táctico | Cuadrantes: 7
[TAC-003] Sensores internos reajustados: giroscopio calibrado.
[BALIZAS] Serie: TAC-003 | Batería: 55% | En línea: sí
[TAC-003] Modelo: Reconocimiento táctico | Cuadrantes: 7
[CAR-001] Sensores internos reajustados: giroscopio calibrado.
[BALIZAS] Serie: CAR-001 | Batería: 70% | En línea: sí
[CAR-001] Modelo: Carga ambiental | Carga: 0.00 / 20.00 kg.
[CAR-002] Energía solicitada: -40%; ajustada a 0%.
[CAR-002] Sensores internos reajustados: giroscopio calibrado.
[BALIZAS] Serie: CAR-002 | Batería: 0% | En línea: no
[CAR-002] Modelo: Carga ambiental | Carga: 0.00 / 35.00 kg.

=== SUPERVISIÓN FINALIZADA ===
BUILD SUCCESSFUL (total time: 0 seconds)
```